Correction TP1

Module: Analyse Numérique **Classes :** $3^{\text{ème}}$ année

TP1: Résolution numérique de systèmes d'équations linéaires

Applications numériques

Soient $n \geq 4$ et $a_i \in \mathbb{R}$ pour $i = 1, 2, 3$.
On s'intéresse à la résolution numérique d'un système linéaire de n équations écrit sous la forme (S_n): $AX = b$ avec

$$A = A(a_1, a_2, a_3) = \begin{pmatrix} a_1 & a_2 & 0 & \dots & \dots & 0 \\ a_3 & a_1 & a_2 & \ddots & & \vdots \\ 0 & a_3 & a_1 & a_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ \vdots & & \ddots & \ddots & \ddots & a_2 \\ 0 & \dots & \dots & 0 & a_3 & a_1 \end{pmatrix}, \quad X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_{n-1} \\ x_n \end{pmatrix} \quad \text{et} \quad b = \begin{pmatrix} 2 \\ 2 \\ \vdots \\ \vdots \\ 2 \\ 2 \end{pmatrix}.$$

```
import numpy as np
import matplotlib.pyplot as plt
```

1. (a) Écrire une fonction `tridiag(a1,a2,a3,n)` qui renvoie la matrice tridiagonale $A = A(a_1, a_2, a_3)$ de taille n et le second membre b du système linéaire (S_n).

```
def tridiag(a1,a2,a3,n):
    A=a1*np.eye(n)+a2*np.diagflat(np.ones(n-1),1)+a3*...
    np.diagflat(np.ones(n-1),-1)
    b=2*np.ones((n,1))
    return A,b
```

(b) Tester la fonction `tridiag(a1,a2,a3,n)` pour $a_1 = 4$, $a_2 = a_3 = 1$ et $n = 10$.

```
a1,a2,a3,n=4,1,1,10
tridiag(a1,a2,a3,n)
```

```
(array([[4., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [1., 4., 1., 0., 0., 0., 0., 0., 0., 0.],
        [0., 1., 4., 1., 0., 0., 0., 0., 0., 0.],
        [0., 0., 1., 4., 1., 0., 0., 0., 0., 0.],
        [0., 0., 0., 1., 4., 1., 0., 0., 0., 0.],
        [0., 0., 0., 0., 1., 4., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 1., 4., 1., 0., 0.],
        [0., 0., 0., 0., 0., 0., 1., 4., 1., 0.],
        [0., 0., 0., 0., 0., 0., 0., 1., 4., 1.],
        [0., 0., 0., 0., 0., 0., 0., 0., 1., 4.]]),
 array([[2.],
        [2.],
        [2.],
        [2.],
        [2.],
        [2.],
        [2.],
        [2.],
        [2.],
        [2.]]))
```

2. (a) Écrire une fonction `matrice_diag_dominante(A)` prenant en entrée une matrice carrée A d'ordre n, qui vérifie si cette matrice est à diagonale strictement dominante ou non.

```
def matrice_diag_dominante(A):
    n=A.shape[0]
    etat=True
    i=0
    while ((i<n) and (etat==True)):
        etat=np.abs(B[i,i])>np.sum(np.abs(A[i,:]))-np.abs(A[i,i])
        i+=1
    return etat
```

(b) Tester la fonction `matrice_diag_dominante(A)` sur les matrices $A_1 = A(4,1,1)$ et $A_2 = A(1,1,1)$ pour $n = 10$.

```
A1=tridiag(4,1,1,10)[0]
A2=tridiag(1,1,1,10)[0]
```

```
matrice_diag_dominante(A1),matrice_diag_dominante(A2)
```

(True, False)

3. (a) Écrire une fonction `jacobi(A, b, X0, epsilon)` prenant en entrée une matrice carrée A d'ordre n, un second membre b, une valeur initiale `X0` et une tolérance `epsilon`, qui renvoie une solution approchée du SEL $AX = b$ par la méthode de Jacobi et le nombre d'itérations effectuées.
La tolérance `epsilon` est utilisée pour le critère d'arrêt: $||AX^{(k)} - b|| \leq$ `epsilon`.
On testera, à l'aide de la fonction `matrice_diag_dominante(A)`, si A est à diagonale strictement dominante, dans le cas contraire, on renverra "A n'est pas à diagonale strictement dominante".

```
def jacobi(A, b, X0, epsilon):
    etat=matrice_diag_dominante(A)
    if  etat==False:
        return ("A n'est pas à diagonale strictement dominante")
    else:
        D=np.diagflat(np.diag(A))
        E=-np.tril(A,-1)
        F=-np.triu(A,1)
        M=D
        N=E+F
        k=0
        while np.linalg.norm(A.dot(X0)-b)>epsilon:
            X0=np.linalg.inv(M).dot(N).dot(X0)+np.linalg.inv(M).dot(b)
            k+=1
        return X0,k
```

(b) En utilisant la fonction `tridiag(a1,a2,a3,n)`, tester la fonction `jacobi`, avec les paramètres et arguments suivants :
$n = 10$, $a1 = 4$, $a2 = a3 = 1$, $X^{(0)} = (1, 1, \ldots, 1)^t \in \mathcal{M}_{10,1}(\mathbb{R})$ et `epsilon` $= 10^{-6}$.

```
n=10
X0=np.ones((n,1))
epsilon=10**-6
A=tridiag(4,1,1,n)[0]
b=tridiag(4,1,1,n)[1]
jacobi(A, b, X0, epsilon)
```

```
(array([[0.42264913],
        [0.30940342],
        [0.33973708],
        [0.33164809],
        [0.33367034],
        [0.33367034],
        [0.33164809],
        [0.33973708],
        [0.30940342],
        [0.42264913]]),
 23)
```

(c) En utilisant la fonction $np.linalg.solve(A, b)$, résoudre (S_{10}) et trouver l'erreur commise par la méthode de Jacobi en norme euclidienne.

```
Sol_exact=np.linalg.solve(A,b)
print(Sol_exact)
E_J=np.linalg.norm(Sol_exact-jacobi(A, b, X0, epsilon)[0])
E_J
```

```
[[0.42264914]
 [0.30940344]
 [0.33973711]
 [0.33164813]
 [0.33367037]
 [0.33367037]
 [0.33164813]
 [0.33973711]
 [0.30940344]
 [0.42264914]]
9.043222924086108e-08
```

4. (a) Écrire une fonction `gauss_seidel(A, b, X0, epsilon)` qui renvoie une solution approchée du SEL $AX = b$ par la méthode de Gauss-Seidel et le nombre d'itérations effectuées en vérifiant si A est à diagonale strictement dominante.

```
def gauss_seidel(A, b, X0, epsilon):
    etat=matrice_diag_dominante(A)
    if  etat==False:
        return ("A n'est pas à diagonale strictement dominante")
    else:
        D=np.diagflat(np.diag(A))
```

```
        E=-np.tril(A,-1)
        F=-np.triu(A,1)
        M=D-E
        N=F
        k=0
        while np.linalg.norm(A.dot(X0)-b)>epsilon:
            X0=np.linalg.inv(M).dot(N).dot(X0)+np.linalg.inv(M).dot(b)
            k+=1
        return X0,k
```

(b) Trouver l'erreur commise par la méthode de Gauss-Seidel en norme euclidienne, avec le même exemple considéré dans 3.b).

```
#sol approchée
print(gauss_seidel(A, b, X0, epsilon))
#Erreur
E_GS=np.linalg.norm(Sol_exact-gauss_seidel(A, b, X0, epsilon)[0])
E_GS
```

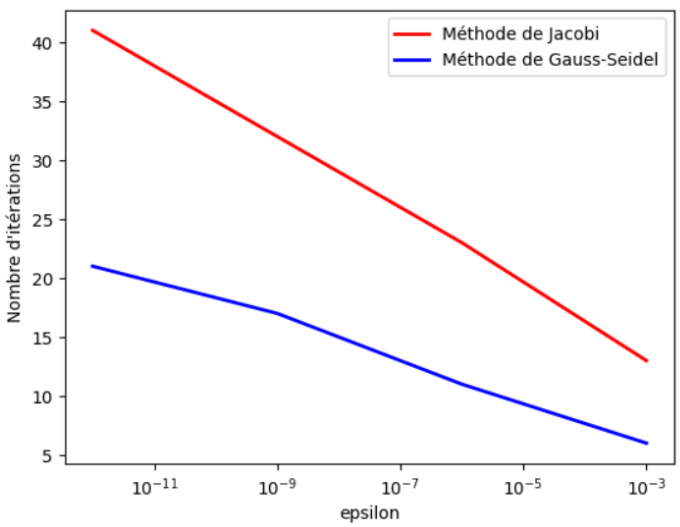
```
(array([[0.42264899],
       [0.30940366],
       [0.3397369 ],
       [0.33164829],
       [0.33367028],
       [0.33367043],
       [0.3316481 ],
       [0.33973712],
       [0.30940343],
       [0.42264914]]), 11)
3.9020162589965853e-07
```

5. Comparer les résultats obtenus par les deux méthodes.
   La méthode de Gauss-Seidel converge plus rapidement que celle de Jacobi, en effet, le nombre maximal d'itérations de Gauss-Seidel ($Niter_{GS} = 11$) est plus petit que le nombre maximal d'itérations de Jacobi ($Niter_J = 23$).
   Par contre la méthode de Jacobi approche mieux la solution du système linéaire (S_{10}) que celle de Gauss-Seidel ($E_J \simeq 9.04 \times 10^{-8}$ et $E_{GS} \simeq 3.9 \times 10^{-7}$).

6. Représenter sur un même graphe, le nombre d'itérations par les deux méthodes itératives : Jacobi et Gauss-Seidel, en fonction de la précision `epsilon`. On considère `epsilon` $\in \{10^{-3}, 10^{-6}, 10^{-9}, 10^{-12}\}$ et $n = 10$. Interpréter les résultats obtenus.

```
#Nombre d'itérations
epsilon=[10**-3,10**-6,10**-9,10**-12]
Iter_J=[]
Iter_GS=[]
for eps in epsilon:
    Iter_J.append(jacobi(A,b,X0,eps)[1])
    Iter_GS.append(gauss_seidel(A,b,X0,eps)[1])

#Représentation graphique
plt.plot(epsilon,Iter_J,'r',epsilon,Iter_GS,'b', linewidth=2,markersize=5)
plt.xscale('log')
plt.xlabel('epsilon',fontsize=10)
plt.ylabel('Nombre dítérations',fontsize=10)
plt.legend(('Méthode de Jacobi','Méthode de Gauss-Seidel'),fontsize=10, loc = 0)
```



Pour $n = 10$, on observe que plus la précision epsilon est petite, plus le nombre maximal d'itérations de Gauss-Seidel et de Jacobi est grand, avec une convergence plus rapide de Gauss-Seidel que celle de Jacobi.

7. (Question Asynchrone) Soit la fonction `Niter(epsilon)` définie comme suit :

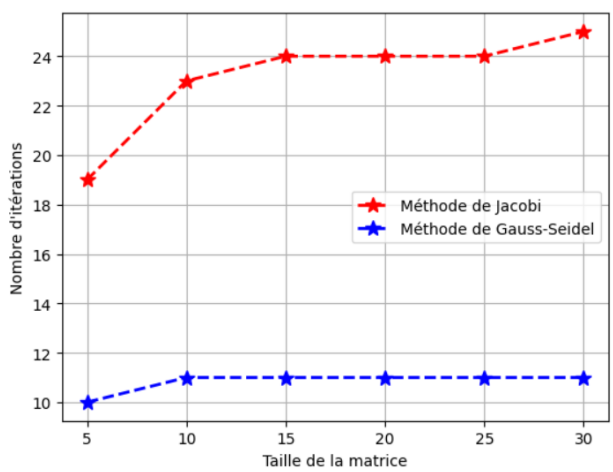
```
def Niter(epsilon):
    N=np.arange(5,31,5)
    Niter_J=[]
    Niter_GS=[]
    for n in N:
```

```
        A=tridiag(4,1,1,n)[0]
        b=tridiag(4,1,1,n)[1]
        X0=np.ones((n,1))
        Niter_J.append(jacobi(A,b,X0,epsilon)[1])
        Niter_GS.append(gauss_seidel(A,b,X0,epsilon)[1])
    return Niter_J, Niter_GS
```

Tester cette fonction pour `epsilon`=10^{-6}. Expliquer les résultats obtenus.

Pour expliquer les résultats obtenus, on pourra représenter sur un même graphe (voir la figure ci-dessous), le nombre d'itérations des deux méthodes itératives en fonction de la taille du système linéaire (S_n).

```
N=np.arange(5,31,5)
epsilon=10**-6
plt.plot(N,Niter(epsilon)[0],'r--',N,Niter(epsilon)[1],'b--', ...
linewidth=2,markersize=5)
plt.legend(('Méthode de Jacobi','Méthode de Gauss-Seidel'),fontsize=10)
plt.xlabel('Taille de la matrice',fontsize=10)
plt.ylabel('Nombre dítérations',fontsize=10)
plt.grid(True)
```



Pour epsilon=10^{-6}, on observe d'une part, que la méthode de Jacobi nécessite plus d'itérations pour converger que celle de Gauss-Seidel, d'autre part le nombre d'itérations de Jacobi croît en augmentant la taille n du système linéaire (S_n) ($n \in \{5, 10, 15, 20, 25, 30\}$), tandis que le nombre d'itérations de Gauss-Seidel reste le même.